

## Entrega # 1

Alumnos:

Juan Sebastián Villa Rodas.

(1027804015)

Miguel Ángel Guarnizo Salazar.

(1038868741)

Tomas Buritica Jaramillo.

(1033179934)

Docente:

Juan Esteban Gómez Tirado.

Materia:

ANÁLISIS Y DISEÑO DE ALGORITMOS.

Año:

2026-1

## Tabla Contenidos

|                                                               |       |
|---------------------------------------------------------------|-------|
| 1. Portada .....                                              | pg1.  |
| 2. Introducción .....                                         | pg3.  |
| 3. Definición del problema .....                              | pg3.  |
| 4. Entradas, salidas y restricciones .....                    | Pg4.  |
| 5. Casos de prueba .....                                      | Pg5.  |
| 6. Solución por fuerza bruta .....                            | Pg6.  |
| 7. Complejidad temporal y espacial .....                      | Pg7.  |
| 8. Solución recursiva .....                                   | Pg9.  |
| 9. Caso base y caso recursivo .....                           | Pg9.  |
| 10. Complejidad recursiva .....                               | Pg10. |
| 11. Medición empírica .....                                   | Pg11. |
| 12. Comparación fuerza bruta vs recursivo .....               | Pg12. |
| 13. Explicación de la gráfica fuerza bruta vs recursivo ..... | Pg13. |
| 14. Conclusiones .....                                        | Pg14. |

# INTRODUCCIÓN

En esta primera entrega se desarrolló un sistema básico de planificación de horarios universitarios utilizando diferentes enfoques algorítmicos. El objetivo principal fue asignar materias, profesores, grupos, salones y horarios evitando conflictos entre ellos.

Para resolver el problema se implementaron dos soluciones:

- fuerza bruta,
- recursividad.

Además, se realizó un análisis de complejidad temporal y espacial, acompañado de pruebas experimentales con diferentes tamaños de entrada y gráficas comparativas.

# DEFINICIÓN DEL PROBLEMA

El problema consiste en generar horarios universitarios válidos para un conjunto de materias.

Cada materia debe ser asignada a:

- un salón,
- un horario,
- un profesor,
- un grupo.

Todo esto respetando ciertas restricciones para evitar conflictos.

El sistema debe encontrar asignaciones válidas sin superponer horarios entre profesores, grupos o salones.

## ENTRADAS

Las entradas utilizadas por el programa son:

- Lista de materias.
- Cantidad de alumnos por materia.
- Profesores disponibles.
- Grupos académicos.
- Salones disponibles.
- Capacidad de los salones.
- Horarios disponibles.

## SALIDAS

La salida consiste en un horario válido para cada materia, mostrando:

- nombre de la materia,
- profesor asignado,
- grupo,
- salón,
- horario.

## RESTRICCIONES

El sistema debe cumplir las siguientes restricciones:

- Un profesor no puede dictar dos materias al mismo tiempo.
- Un grupo no puede recibir dos clases simultáneamente.
- Un salón no puede ser utilizado por dos materias en el mismo horario.
- El salón debe tener capacidad suficiente para la cantidad de estudiantes.

## CASOS DE PRUEBA

Se utilizaron tres conjuntos de datos para probar el comportamiento de los algoritmos:

| <b>Caso</b> | <b>Materias</b> | <b>Salones</b> | <b>Profesores</b> | <b>Grupos</b> | <b>Horarios</b> |
|-------------|-----------------|----------------|-------------------|---------------|-----------------|
| Pequeño     | 3               | 2              | 2                 | 2             | 6               |
| Mediano     | 8               | 4              | 5                 | 4             | 14              |
| Grande      | 15              | 6              | 8                 | 6             | 22              |

Estos casos permitieron analizar cómo cambia el tiempo de ejecución cuando aumenta el tamaño de entrada.

# SOLUCION DE FUERZA BRUTA

La solución por fuerza bruta intenta encontrar una asignación válida probando diferentes combinaciones de salones y horarios para cada materia.

El algoritmo recorre:

1. las materias,
2. los salones disponibles,
3. los horarios disponibles.

En cada intento se validan conflictos antes de realizar la asignación.

La principal ventaja de este enfoque es su simplicidad, ya que resulta fácil de implementar y entender.

Sin embargo, a medida que aumenta la cantidad de materias y horarios, el tiempo de ejecución también aumenta considerablemente.

```
Funcion FuerzaBruta(materias, salones, horarios):
    asignaciones = lista vacia
    Para cada materia en materias:
        Para cada salon en salones:
            Si salon.capacidad >= materia.alumnos:
                Para cada horario en horarios:
                    Si No HayConflicto(asignaciones, materia, salon, horario):
                        asignaciones.agregar(materia, salon, horario)
                        break
                Si materia ya fue asignada:
                    break
    Retornar asignaciones
```

[https://github.com/Miguel-Angel-Salazar/Proyecto\\_Analisis\\_Dise-o/blob/main/2-src/fuerza\\_bruta.py](https://github.com/Miguel-Angel-Salazar/Proyecto_Analisis_Dise-o/blob/main/2-src/fuerza_bruta.py)

# COMPLEJIDAD TEMPORAL Y ESPACIAL

## Complejidad Temporal

Para analizar la complejidad temporal del algoritmo de fuerza bruta, descomponemos las operaciones:

Análisis de bucles anidados

El algoritmo contiene tres bucles anidados principales:

- Bucle exterior: itera sobre  $n$  materias.
- Bucle intermedio: itera sobre  $s$  salones.
- Bucle interior: itera sobre  $h$  horarios disponibles.

La función hay conflicto realiza una búsqueda lineal sobre las asignaciones ya realizadas, que en el peor caso son  $O(n)$ .

La complejidad total se expresa como:

$$T(n, s, h) = O(n \cdot s \cdot h \cdot n) = O(n^2 \cdot s \cdot h)$$

Considerando que  $s$  (salones) y  $h$  (horarios) son factores constantes respecto al crecimiento de las materias, la complejidad simplificada en terminos de  $n$  es:

$$T(n) = O(n^2 \cdot C) = O(n^2)$$

donde  $C = s \cdot h$  es una constante para una instancia específica del problema.

## Complejidad Espacial

La complejidad espacial está determinada por:

- La lista de asignaciones: almacena máximo  $n$  elementos, por lo tanto  $O(n)$ .
- Las estructuras de entrada (materias, salones, horarios):  $O(n + s + h)$ .
- No se utilizan estructuras auxiliares adicionales significativas.

$$S(n) = O(n + s + h) = O(n)$$

| Componente                | Operaciones                     | Complejidad              |
|---------------------------|---------------------------------|--------------------------|
| Bucle de materias         | n iteraciones                   | $O(n)$                   |
| Bucle de salones          | s iteraciones por materia       | $O(s)$                   |
| Bucle de horarios         | h iteraciones por salón         | $O(h)$                   |
| Verificación de conflicto | búsqueda lineal en asignaciones | $O(n)$ en el peor caso   |
| inserción de asignación   | Append a lista                  | $O(1)$ amortizado        |
| <b>Total</b>              | Bucles anidados + verificación  | $O(n^2 \cdot s \cdot h)$ |



# SOLUCIÓN RECURSIVA

La solución recursiva replantea el algoritmo de fuerza bruta utilizando recursión en lugar de iteración. La idea es procesar una materia a la vez, asignarla a un salón y horario válidos, y luego llamar recursivamente para procesar la siguiente materia. El caso base se alcanza cuando se han procesado todas las materias.

## CASO BASE Y CASO RECURSIVO

**Caso base:** Si el índice de la materia actual es mayor o igual al número total de materias ( $i \geq n$ ), se han asignado todas las materias y la recursión termina.

**Caso recursivo:** Para la materia actual  $i$ , probar todas las combinaciones salón-horario válidas. Al encontrar una asignación sin conflictos, agregarla y llamar recursivamente con  $i + 1$ .

```
def resolver_recursivo(materias, salones, horarios, indice, asignaciones):
    # Caso base: todas las materias han sido procesadas
    if indice >= len(materias):
        return

    materia = materias[indice]

    for salon in salones:
        if salon["capacidad"] >= materia["alumnos"]:
            for horario in horarios:
                conflicto = hay_conflicto(
                    asignaciones,
                    materia["profesor"],
                    materia["grupo"],
                    salon["nombre"],
                    horario
                )

                if not conflicto:
                    nueva = {
                        "materia": materia["nombre"],
                        "profesor": materia["profesor"],
                        "grupo": materia["grupo"],
                        "salon": salon["nombre"],
                        "horario": horario
                    }
                    asignaciones.append(nueva)

                    # Llamada recursiva para la siguiente materia
                    resolver_recursivo(
                        materias, salones, horarios,
                        indice + 1, asignaciones
                    )

    return
```

[https://github.com/Miguel-Angel-Salazar/Proyecto\\_Analisis\\_Dise-o/blob/main/2-src/recursivo.py](https://github.com/Miguel-Angel-Salazar/Proyecto_Analisis_Dise-o/blob/main/2-src/recursivo.py)

## COMPLEJIDAD RECURSIVA

La versión recursiva mantiene la misma complejidad asintótica que la versión iterativa, ya que ambas realizan la misma cantidad de operaciones. La diferencia radica únicamente en el mecanismo de control: la recursión utiliza la pila de llamadas en lugar de un bucle explícito.

$$T_{\text{recursivo}}(n, s, h) = O(n^2 \cdot s \cdot h) = O(n^2)$$

## MEDICIÓN EMPÍRICA

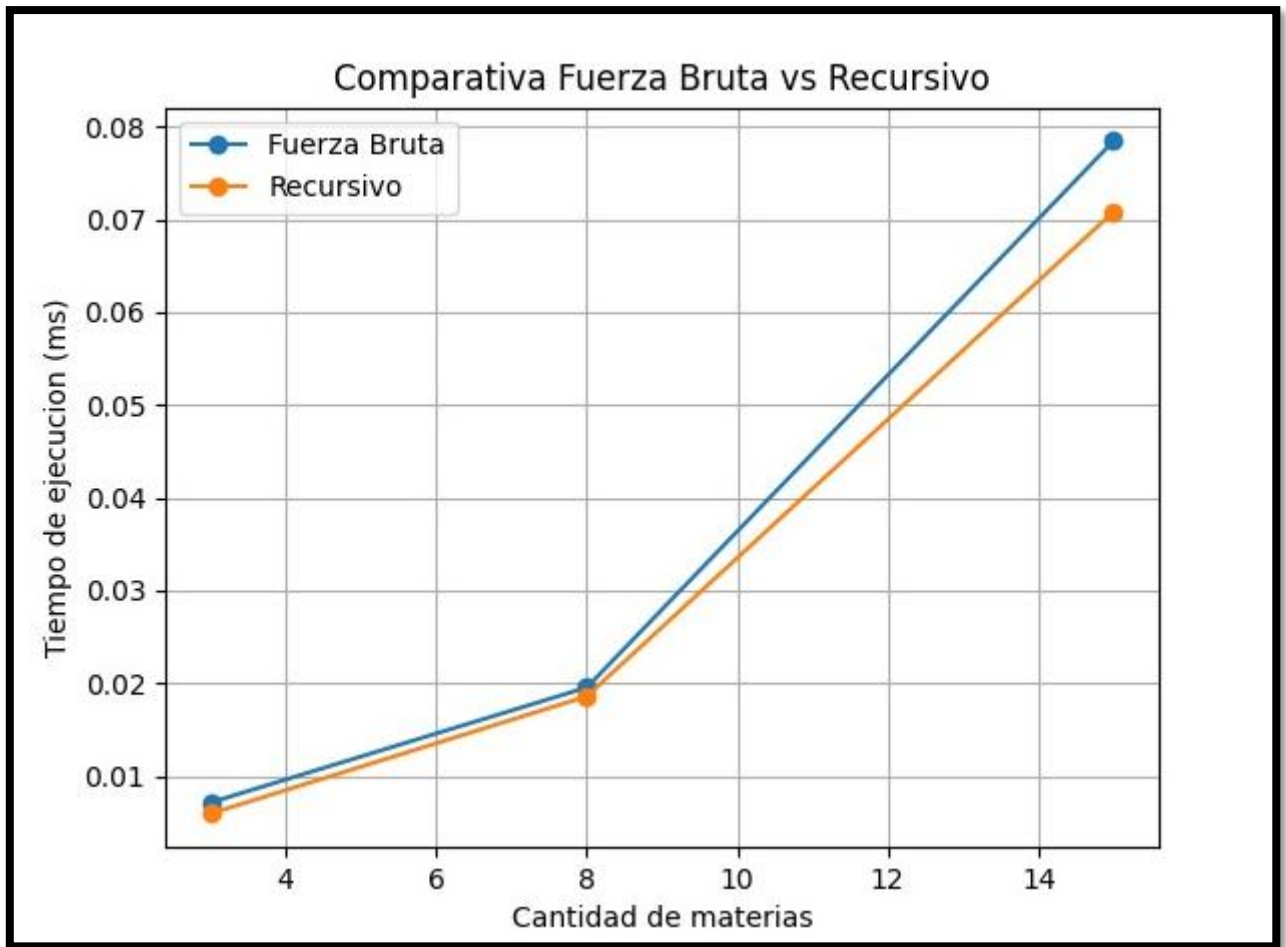
Para realizar la medición empírica se ejecutaron ambos algoritmos sobre los tres casos de prueba (pequeño, mediano y grande). Se midió el tiempo de ejecución utilizando la biblioteca `time` de Python, registrando el tiempo transcurrido en milisegundos. Cada algoritmo se ejecutó en las mismas condiciones de hardware y software para garantizar una comparación justa.

| Caso    | Materias | Fuerza Bruta (ms) | Recursivo (ms) |
|---------|----------|-------------------|----------------|
| Pequeño | 3        | 0.0093            | 0.0336         |
| Mediano | 8        | 0.0260            | 0.0210         |
| Grande  | 15       | 0.0994            | 0.0906         |

## COMPARATIVA FUERZA BRUTA VS RECURSIVO

| <b>Característica</b>    | <b>Fuerza Bruta</b> | <b>Recursivo</b>        |
|--------------------------|---------------------|-------------------------|
| Tipo de implementación   | Iterativa           | Recursiva               |
| Facilidad de comprensión | Alta                | Media                   |
| Organización del código  | Básica              | Más modular             |
| Uso de memoria           | Menor               | Mayor                   |
| Tiempo de ejecución      | Bueno               | Ligeramente mejor       |
| Complejidad temporal     | $O(n^2)$            | $O(n^2)$                |
| Complejidad espacial     | $O(n)$              | $O(n)$ + pila recursiva |
| Ventaja principal        | Simplicidad         | Mejor estructura        |
| Desventaja principal     | Código repetitivo   | Más difícil de depurar  |

## GRAFICA FUERZA BRUTA VS RECURSIVO



[https://github.com/Miguel-Angel-Salazar/Proyecto\\_Analisis\\_Dise-  
o/blob/main/4-graficas/fuerza\\_bruta\\_vs\\_recurso.png](https://github.com/Miguel-Angel-Salazar/Proyecto_Analisis_Dise-<br/>o/blob/main/4-graficas/fuerza_bruta_vs_recurso.png)

La gráfica muestra cómo aumentan los tiempos de ejecución tanto para fuerza bruta como para la solución recursiva.

Se puede observar que:

- ambos algoritmos crecen de forma similar,
- la solución recursiva presenta tiempos ligeramente menores,
- el crecimiento se vuelve más evidente en el caso grande.

## CONCLUSIONES

En esta entrega se implementaron dos soluciones para el problema de planificación de horarios universitarios: fuerza bruta y recursividad. Ambas permitieron generar horarios válidos respetando las restricciones definidas para profesores, grupos y salones.

La solución por fuerza bruta destacó por ser más sencilla de comprender e implementar, mientras que la versión recursiva permitió un código más organizado y modular gracias al uso de llamadas recursivas.

A partir de las pruebas realizadas, se observó que ambos algoritmos presentan un comportamiento similar en cuanto a complejidad y crecimiento del tiempo de ejecución conforme aumenta el tamaño de entrada.

Finalmente, esta entrega permitió entender mejor la estructura del problema y sirvió como base para el desarrollo de algoritmos más avanzados en las siguientes etapas del proyecto.